

UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3067 – Redes de Computadoras

Sección 10

Ing. Kevin Velásquez



Laboratorio 2 - Detección y Corrección de Errores en L2 OSI

Dulce Ambrosio (231143)

Juan Cruz (23110)

Guatemala, 02 de agosto de 2026

Introducción

El objetivo de esta práctica fue implementar y comparar dos esquemas de manejo de errores de transmisión, uno de corrección (código de Hamming) y uno de detección (CRC-32), integrándolos en una arquitectura de capas que simula el modelo de una comunicación de datos real.

La arquitectura contempla las capas de Aplicación, Presentación, Enlace, Ruido y Transmisión. El programa emisor se implementó en Python y el receptor en C++, la comunicación entre ambos programas se realiza mediante sockets TCP reales, no se simula ni se copian los datos a mano, aunque el receptor también admite un modo de entrada por consola (--stdin) para pegar tramas manualmente si se desea reproducir un escenario sin levantar el socket.

Arquitectura de capas implementada

Algoritmo de corrección:

Código de Hamming, se implementó de forma general para cualquier código (n,m) , donde m es la cantidad de bits de datos por bloque (configurable, por defecto 4, es decir Hamming(7,4)) y r se calcula automáticamente como el mínimo número de bits de paridad que satisface la desigualdad. El mensaje se divide en bloques de m bits, se agrega padding de ceros al último bloque si es necesario, y cada bloque se codifica de forma independiente.

Algoritmo de detección:

CRC-32, se utilizó el polinomio estándar de IEEE 802.3 (0x04C11DB7, representado en su forma reflejada como 0xEDB88320), con valor inicial 0xFFFFFFFF y XOR final 0xFFFFFFFF, los mismos parámetros que usan Ethernet, ZIP y zlib. El checksum de 32 bits se calcula sobre todo el mensaje y se concatena al final de la trama.

Protocolo de transmisión:

Sobre el socket se envían dos líneas de texto, un header con el algoritmo y sus parámetros y la cadena de bits de la trama ya con el ruido aplicado. Esto permite que el receptor sepa qué algoritmo usar y dónde separa los datos de la redundancia, sin necesidad de negociarlo por separado.

Capa	Emisor (Python)	Receptor (C++)
Aplicación	solicitar_mensaje	mostrar_mensaje
Presentación	codificar_mensaje	decodificar_mensaje
Enlace	calcular_integridad	verificar_integridad, corregir_mensaje
Ruido	aplicar_ruido/forzar_flips	-
Transmisión	enviar_informacion	recibir_informacion

Resultados

Se probó cada algoritmo con tres escenarios, sin errores, con un solo bit de error, y con dos o más bits de error en el mismo bloque. Para que los escenarios fueran reproducibles se usó la opción --forzar-flips del emisor, que invierte bits en posiciones exactas de la trama en lugar de depender del ruido aleatorio.

CRC-32 — Prueba 1: sin errores

```
PS C:\Users\Dulce\OneDrive - UVG\UG\8 Semestre\Redes\CC6037_REDES\receptor> .\receptor.exe 5000
[TRANSMISION] Receptor escuchando en el puerto 5000

[TRANSMISION] Header: CRC32:72
[TRANSMISION] Trama recibida (104 bits): 010100100110010101100100011001010111001100100000010101010101100100011101110101000010001101001000000101
[ENLACE] CRC-32 -> checksum correcto
[APLICACION] Mensaje recibido sin errores: "Redes UVG"

PS C:\Users\Dulce\OneDrive - UVG\UG\8 Semestre\Redes\CC6037_REDES\emisor> python emisor.py --mensaje "Redes UVG" --algoritmo crc32 --tasa-error 0 --port 5000
[APLICACION] Mensaje: 'Redes UVG' | Algoritmo: crc32
[PRESENTACION] Binario ASCII (72 bits): 0101001001100101011001000110010101110011001000000101010101011001000111
[ENLACE] CRC-32 -> 104 bits (72 de datos + 32 de checksum)
[ENLACE] Trama con integridad: 010100100110010101100100011001010111001100100000010101010101100100011101110101000010001101001000000101
[RUIDO] Probabilidad 0.0 -> 0 bit(s) invertido(s)
[RUIDO] Trama transmitida: 010100100110010101100100011001010111001100100000010101010101100100011101110101000010001101001000000101
[TRANSMISION] Enviado a 127.0.0.1:5000 con header 'CRC32:72'

PS C:\Users\Dulce\OneDrive - UVG\UG\8 Semestre\Redes\CC6037_REDES\emisor> 
```

CRC-32 — Prueba 2: 1 bit de error

```
[TRANSMISION] Header: CRC32:72
[TRANSMISION] Trama recibida (104 bits): 0101011001100101011001000110010101110011001000000101010101100100011101110101000010001101001000000101
[ENLACE] CRC-32 -> checksum incorrecto
[APLICACION] Error detectado, no se pudo corregir. Trama descartada.

PS C:\Users\Dulce\OneDrive - UVG\UVG\8 Semestre\Redes\CC6037_REDES\emisor> python emisor.py --mensaje "Redes UVG" --algoritmo crc32 --forzar-flips 5 --port 5000

[APLICACION] Mensaje: 'Redes UVG' | Algoritmo: crc32
[PRESENTACION] Binario ASCII (72 bits): 01010010011001010110010001100101011100110010000001010101011001000111
[ENLACE] CRC-32 -> 104 bits (72 de datos + 32 de checksum)
[ENLACE] Trama con integridad: 0101001001100101011001000110010101110011001000000101010101100100011101110101000010001101001000000101
[RUIDO] Bits invertidos manualmente en: [5]
[RUIDO] Trama transmitida: 01010110011001010110010001100101011100110010000001010101011001000111011101000010001101001000000101
[TRANSMISION] Enviado a 127.0.0.1:5000 con header 'CRC32:72'
```

CRC-32 — Prueba 3: 2 bits de error

```
[TRANSMISION] Header: CRC32:72
[TRANSMISION] Trama recibida (104 bits): 0101011001100101011001000110010101110011101000000101010101100100011101110101000010001101001000000101
[ENLACE] CRC-32 -> checksum incorrecto
[APLICACION] Error detectado, no se pudo corregir. Trama descartada.

PS C:\Users\Dulce\OneDrive - UVG\UVG\8 Semestre\Redes\CC6037_REDES\emisor> python emisor.py --mensaje "Redes UVG" --algoritmo crc32 --forzar-flips 5,40 --port 5000

[APLICACION] Mensaje: 'Redes UVG' | Algoritmo: crc32
[PRESENTACION] Binario ASCII (72 bits): 01010010011001010110010001100101011100110010000001010101011001000111
[ENLACE] CRC-32 -> 104 bits (72 de datos + 32 de checksum)
[ENLACE] Trama con integridad: 0101001001100101011001000110010101110011001000000101010101100100011101110101000010001101001000000101
[RUIDO] Bits invertidos manualmente en: [5, 40]
[RUIDO] Trama transmitida: 01010110011001010110010001100101011100111010000001010101011001000111011101000010001101001000000101
[TRANSMISION] Enviado a 127.0.0.1:5000 con header 'CRC32:72'
```

Hamming(7,4) — Prueba 1: sin errores

```
[TRANSMISION] Header: HAMMING:4:3:72
[TRANSMISION] Trama recibida (126 bits): 01001010101011001100100111001101001100110011001001000111110000110101000000001001010010100101110011010011000001111
[ENLACE] Hamming(7,4) -> 0 bloque(s) corregido(s), 0 irrecuperable(s)
[APLICACION] Mensaje recibido sin errores: "Redes UVG"

PS C:\Users\Dulce\OneDrive - UVG\UVG\8 Semestre\Redes\CC6037_REDES\emisor> python emisor.py --mensaje "Redes UVG" --algoritmo hamming --m 4 --tasa-error 0 --port 5000

[APLICACION] Mensaje: 'Redes UVG' | Algoritmo: hamming
[PRESENTACION] Binario ASCII (72 bits): 01010010011001010110010001100101011100110010000001010101011001000111
[ENLACE] Hamming(7,4) -> 18 bloques, 126 bits de trama
[ENLACE] Trama con integridad: 010010101010110011001001100110011001100100100011110000110101000000001001010010100101110011010011000001111
[RUIDO] Probabilidad 0.0 -> 0 bit(s) invertido(s)
[RUIDO] Trama transmitida: 01001010101011001100100110011001100110011001001000111100001101010000000001001010010100101110011010011000001111
[TRANSMISION] Enviado a 127.0.0.1:5000 con header 'HAMMING:4:3:72'
```

Hamming(7,4) — Prueba 2: 1 bit de error

```
PS C:\Users\Dulce\OneDrive - UVG\UVG\8 Semestre\Redes\CC6037_REDES\emisor> python emisor.py --mensaje "Redes UVG" --algoritmo hamming --m 4 --forzar-flips 2 --port 5000

[APLICACION] Mensaje: 'Redes UVG' | Algoritmo: hamming
[PRESENTACION] Binario ASCII (72 bits): 01010010011001010110010001100101011100110010000001010101011001000111
[ENLACE] Hamming(7,4) -> 18 bloques, 126 bits de trama
[ENLACE] Trama con integridad: 010010101010110011001001100110011001100100100011110000110101000000001001010010100101110011010011000001111
[RUIDO] Bits invertidos manualmente en: [2]
[RUIDO] Trama transmitida: 01101010101011001100100110011001100110011001001000111100001101010000000001001010010100101110011010011000001111
[TRANSMISION] Enviado a 127.0.0.1:5000 con header 'HAMMING:4:3:72'

PS C:\Users\Dulce\OneDrive - UVG\UVG\8 Semestre\Redes\CC6037_REDES\emisor>

[TRANSMISION] Header: HAMMING:4:3:72
[TRANSMISION] Trama recibida (126 bits): 0110101010101100110010011001100110011001100100100011110000110101000000001001010010100101110011010011000001111
[ENLACE] Hamming(7,4) -> 1 bloque(s) corregido(s), 0 irrecuperable(s)
[APLICACION] Errores corregidos en 1 bloque(s). Mensaje recuperado: "Redes UVG"
```

Hamming(7,4) — Prueba 3: 2 bits de error en el mismo bloque

```
[TRANSMISION] Header: HAMMING:4:3:72
[TRANSMISION] Trama recibida (126 bits): 1000101010101100110010011100110011001100110010010001111000011010100000000100101001010101110011010011000001111
[ENLACE] Hamming(7,4) -> 1 bloque(s) corregido(s), 0 irrecuperable(s)
[APLICACION] Errores corregidos en 1 bloque(s). Mensaje recuperado: "\x02edes UVG"
```

```

PS C:\Users\Dulce\OneDrive - UVG\UVG\8 Semestre\Redes\CC6037_REDES\emisor> python emisor.py --mensaje "Redes UVG" --algoritmo hamming --m 4 --forzar-flips 0,1 --port 5000

[APLICACION] Mensaje: 'Redes UVG' | Algoritmo: hamming
[PRESENTACION] Binario ASCII (72 bits): 0101001001100101011001000110010101110011001000000101010101011001000111
[ENLACE] Hamming(7,4) -> 18 bloques, 126 bits de trama
[ENLACE] Trama con integridad: 0100101010101100110011001100110011001100111100001101010100000000100101001010101110011010011000001111
[RUIDO] Bits invertidos manualmente en: [0, 1]
[RUIDO] Trama transmitida: 1000101010101100110011001100110011001100111100001101010100000000100101010010101110011010011000001111
[TRANSMISION] Enviado a 127.0.0.1:5000 con header 'HAMMING:4:3:72'

PS C:\Users\Dulce\OneDrive - UVG\UVG\8 Semestre\Redes\CC6037_REDES\emisor>

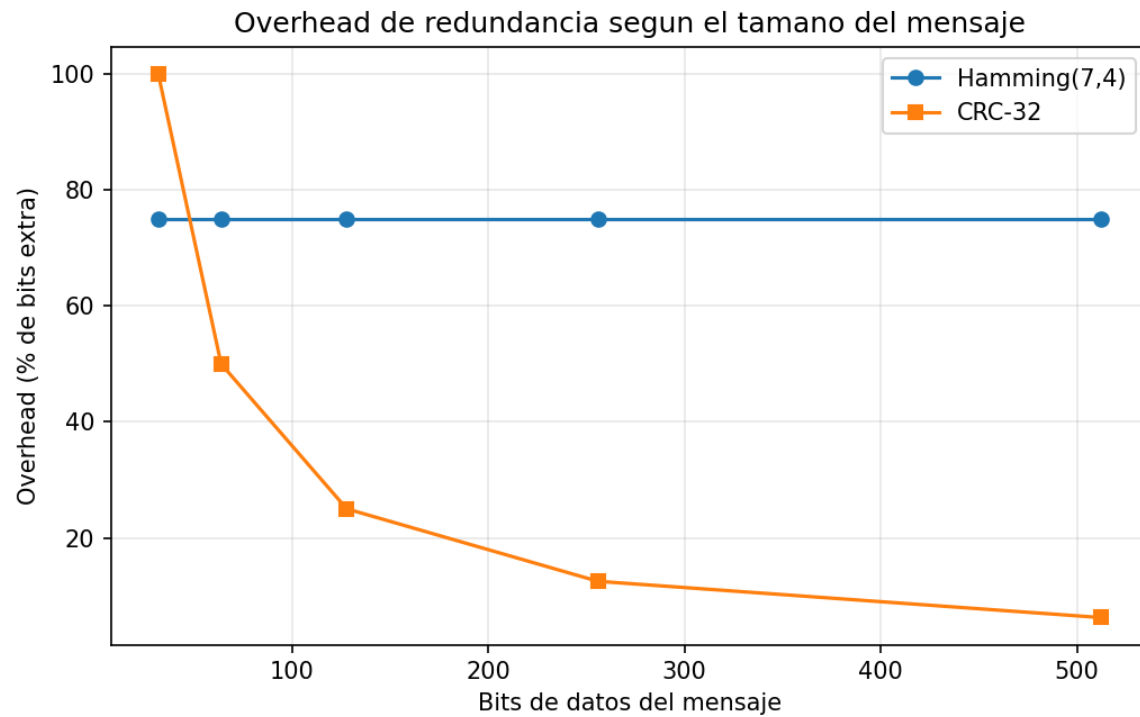
```

En la prueba 3 de Hamming el receptor no detecta que algo salió mal, reporta "1 bloque corregido" y entrega un mensaje distinto al original ("xD2edes UVG" en vez de "Redes UVG"). Esto sucede por que dos errores en el mismo bloque de Hamming(7,4) mueven el síndrome a una posición válida distinta, y el algoritmo corrige el bit equivocado sin darse cuenta.

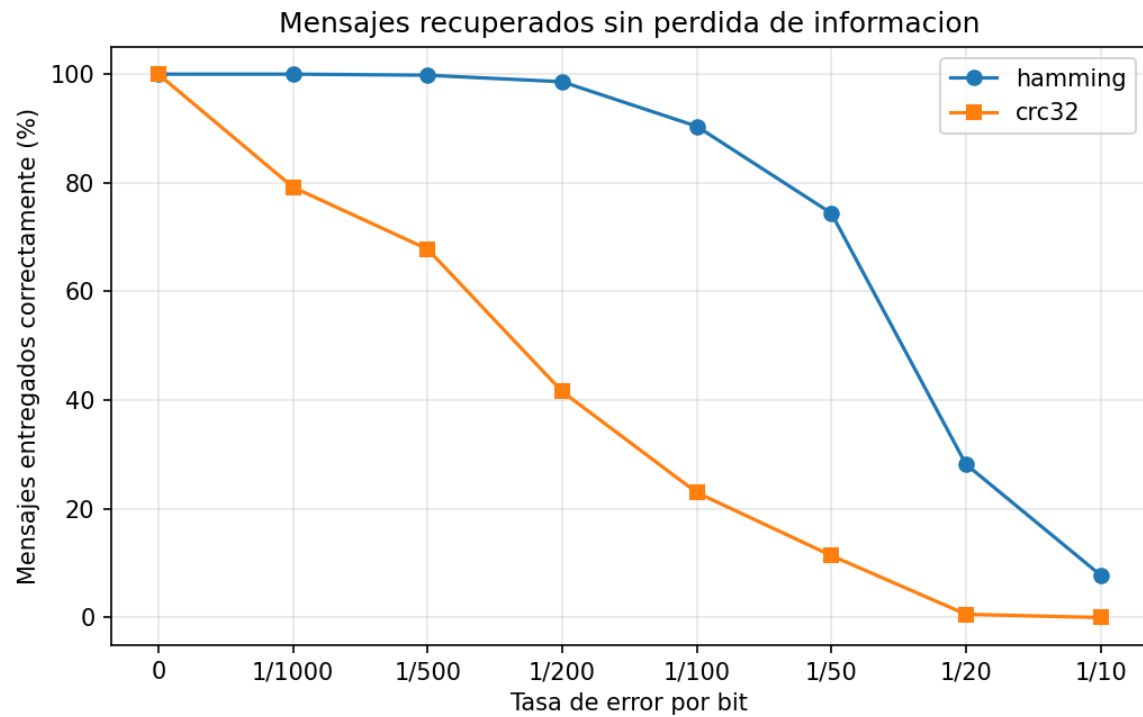
Pruebas estadísticas

Se realizó un script (pruebas.py) que genera mensajes aleatorios de 5 longitudes distintas (4, 8, 16, 32 y 64 caracteres), los codifica con ambos algoritmos, les aplica ruido con 8 tasas de error distintas (desde 0 hasta 1/10) y pasa cada trama resultante por el receptor de C++ real (usando su modo --stdin), repitiendo cada combinación 100 veces. En total se procesaron 8,000 tramas. Los resultados completos están en resultados/resultados.csv; a continuación, se resume una muestra representativa con mensajes de 32 caracteres (256 bits de datos):

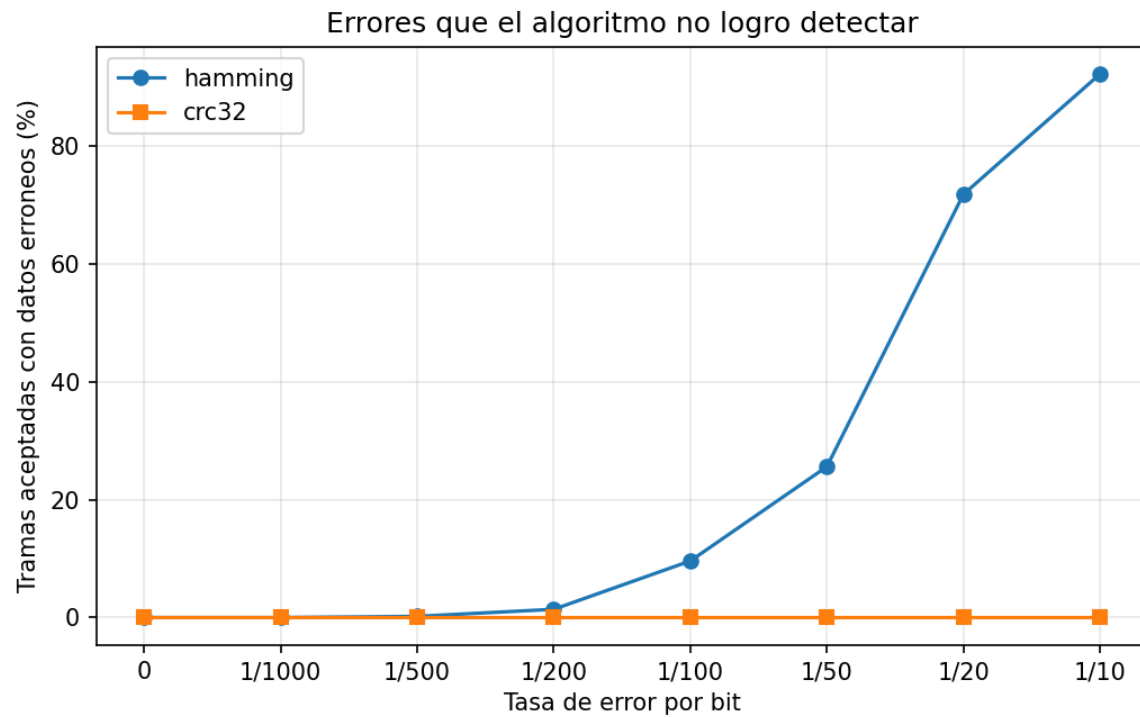
Algoritmo	Tasa de error	Entregadas correctas	Descartadas	Entregas erróneas
CRC-32	0	100%	0%	0%
CRC-32	1/100	5%	95%	0%
CRC-32	1/20	1%	99%	0%
Hamming(7,4)	0	100%	0%	0%
Hamming(7,4)	1/100	86%	0%	14%
Hamming(7,4)	1/20	65%	0%	35%



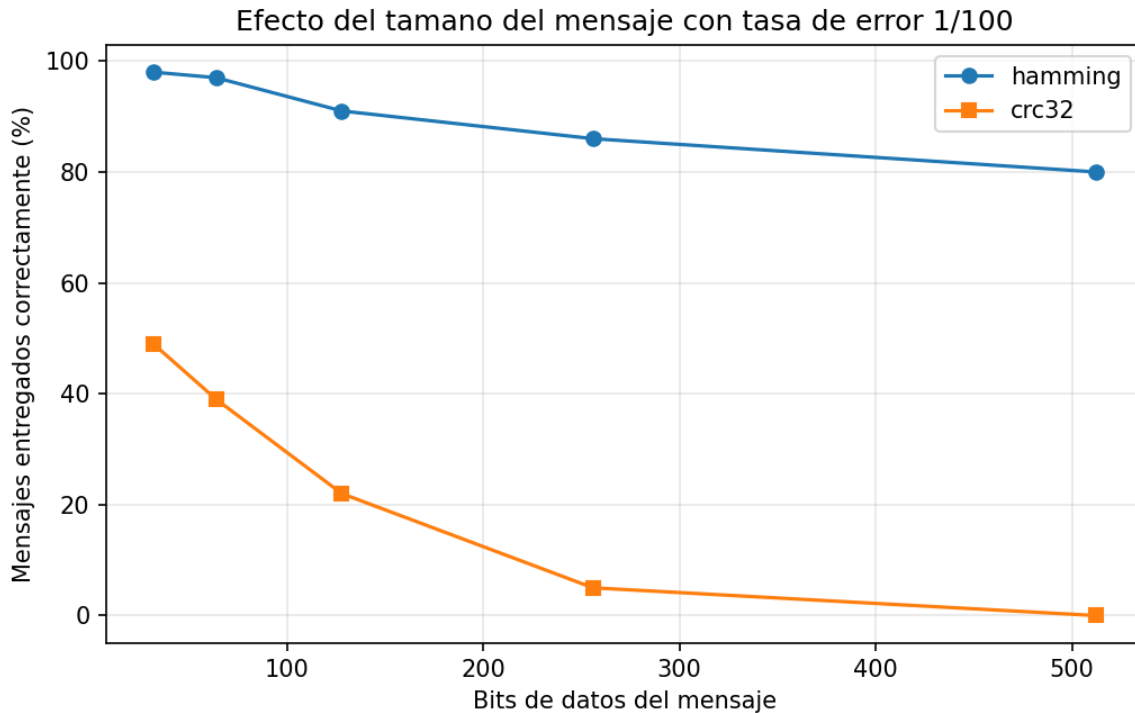
El overhead de Hamming(7,4) es constante (75%, es decir 3 bits de paridad por cada 4 de datos) sin importar el tamaño del mensaje, porque el mismo bloque de 7 bits se repite. El overhead de CRC-32, en cambio, es fijo en tamaño absoluto (siempre 32 bits) pero decrece porcentualmente conforme el mensaje crece de 100% para 4 caracteres a solo 6.25% para 64 caracteres.



Hamming logra recuperar el mensaje correcto con más frecuencia que CRC-32 en casi todo el rango de tasas de error, porque además de detectar corrige. Sin embargo, esta gráfica sola es engañosa, no distingue entre mensajes corregidos de verdad y mensajes corruptos que Hamming entregó creyendo que estaban bien.



Esta gráfica es la más importante del reporte, CRC-32 se mantiene en 0% de errores no detectados en todo el rango de tasas probado, mientras que Hamming empieza a fallar silenciosamente a partir de tasas de error medias-altas (1/200 en adelante) y llega a entregar mensajes corruptos en más del 90% de los casos con una tasa de error de 1/10. Esto ocurre porque, a mayor tasa de error, es más probable que caigan dos o más errores en el mismo bloque de 7 bits, y Hamming(7,4) solo puede garantizar la corrección de un error por bloque.



Con una tasa de error fija, mensajes más largos tienen más bits expuestos al canal y por lo tanto más probabilidad acumulada de sufrir al menos un error; ambos algoritmos empeoran conforme el mensaje crece, pero CRC-32 empeora mucho más rápido porque cualquier error en cualquier parte de un mensaje largo hace que la trama completa se descarte, mientras que Hamming solo pierde el bloque de 7 bits afectado.

Discusión

Depende de qué se entienda por funcionar bien, si el criterio es maximizar el porcentaje de mensajes que el receptor entrega como válidos, Hamming(7,4) gana en casi todo el rango de tasas de error probado. Pero si el criterio es confiabilidad que un mensaje entregado realmente sea el mensaje correcto CRC-32 es mejor, en las 8,000 tramas de la prueba nunca aceptó una trama corrupta como válida, mientras que Hamming lo hizo con una frecuencia creciente. En ese sentido, CRC-32 tuvo el mejor funcionamiento como mecanismo de integridad, aunque a costa de descartar y no de recuperar la información dañada.

Hamming(7,4) tolera tasas de error más altas sin descartar mensajes, porque corrige en vez de solo detectar, hasta 1/200 recupera el 100% de los mensajes de 32 caracteres en las pruebas, y sigue entregando más del 85% de forma correcta

en 1/100. Sin embargo, esa flexibilidad tiene un costo oculto, a partir de cierto punto ya no está corrigiendo de verdad, sino fallando en silencio. CRC-32 no es flexible en el mismo sentido ante cualquier error simplemente descarta la trama, pero es mucho más honesto respecto a cuándo el canal está fallando.

Es mejor utilizar un algoritmo de detección en lugar de uno de corrección cuando:

- Cuando existe la posibilidad de pedir retransmisión, basta con detectar el error y descartar la trama, porque el emisor puede reenviarla; agregar redundancia de corrección sería overhead innecesario
- Cuando el mensaje es largo o el ancho de banda es limitado, el overhead de CRC-32 cae por debajo del 10% para mensajes de más de 40 caracteres, mientras que Hamming(7,4) siempre cuesta 75% adicional
- Cuando es preferible perder un mensaje a aceptar uno corrupto sin saberlo, como en integridad de archivos o transacciones, donde un dato silenciosamente incorrecto es peor que un dato faltante.
- En cambio, conviene usar corrección cuando la retransmisión es costosa o imposible y se prefiere un mensaje parcialmente reparado a ninguno, siempre que la tasa de error se mantenga dentro de la capacidad de corrección del código.

Sí es posible engañar al algoritmo, se demostró en la Prueba 3 de Hamming, al invertir dos bits dentro del mismo bloque de 7 bits (posiciones 0 y 1 de la trama), el receptor calculó un síndrome que apuntaba a una tercera posición, corrigió ese bit que en realidad estaba bien y entregó el mensaje "xD2edes UVG" reportando además que había corregido exitosamente un error. La debilidad estructural es que Hamming(7,4) tiene distancia mínima 3, lo cual garantiza corregir hasta 1 error o detectar hasta 2, pero no ambas cosas a la vez, si ocurren exactamente 2 errores en un bloque, el código los interpreta como si fueran 1 error en otra posición y corrige mal, sin ninguna señal de alarma. CRC-32, en cambio, no se pudo engañar en ninguna de las 8,000 tramas probadas su polinomio de grado 32 hace que la probabilidad de que un patrón de error arbitrario sea invisible al checksum.

Análisis comparativo

Aspecto	Hamming(7,4)	CRC-32
Tipo	Corrección	Detección
Redundancia	75% (constante, 3 bits por cada 4 de datos)	32 bits fijos (cae al crecer el mensaje)
Dificultad de implementación	Media alta (posiciones de paridad, síndromes por bloque)	Baja media (tabla precalculada, muy documentado)
Velocidad	Más lenta (procesa bloque por bloque de 7 bits)	Rápida (tabla de 256 entradas, un byte a la vez)
Ante 1 error	Corrige exactamente	Detecta y descarta
Ante 2+ errores en un bloque	Puede fallar en silencio (falso positivo)	Prácticamente siempre lo detecta

Comentario grupal

Este laboratorio nos ayudó a entender que detectar y corregir errores no son intercambiables, sino que responden a necesidades distintas, nos sorprendió ver en las pruebas que Hamming, a pesar de tener mejor tasa de entrega, puede fallar en silencio y entregar un mensaje corrupto sin avisar, mientras que CRC-32 nunca se equivocó pero tampoco recupera nada por sí solo. Trabajar con dos lenguajes distintos comunicados por sockets también nos obligó a pensar en el protocolo como algo explícito y no como una estructura de datos interna, que fue lo más interesante de la parte de implementación.

Conclusiones

- No existe un algoritmo mejor, Hamming maximiza mensajes entregados pero puede fallar en silencio, mientras que CRC-32 es más confiable como mecanismo de integridad pero no recupera datos por sí solo.
- El overhead de un esquema de corrección de bloque fijo (como Hamming(7,4)) no escala con el tamaño del mensaje, mientras que el de un checksum de longitud fija (como CRC-32) se vuelve proporcionalmente más barato entre más grande es el mensaje.
- La capacidad de corrección de Hamming está limitada por su distancia mínima, un código que garantiza corregir 1 error no puede, al mismo tiempo, garantizar detectar 2 errores en el mismo bloque sin arriesgarse a una corrección equivocada.
- Probar únicamente los 3 escenarios manuales (0, 1 y 2+ errores) alcanza para verificar que la implementación es correcta, pero solo con pruebas a mayor escala (miles de tramas variando longitud y tasa de error) se hace evidente el problema de las correcciones silenciosas de Hamming, que en una prueba pequeña podría pasar desapercibido.
- Implementar el emisor y el receptor en lenguajes distintos, comunicados por sockets reales, obligó a definir un protocolo de trama explícito en vez de depender de estructuras de datos propias de un solo lenguaje, lo cual refleja mejor cómo interoperan sistemas reales en una red.

Referencias

- Hamming, R. W. (1950). Error detecting and error correcting codes. Bell System Technical Journal, 29(2), 147–160. <https://doi.org/10.1002/j.1538-7305.1950.tb00463.x>
- IEEE Standards Association. (2022). IEEE 802.3-2022: Standard for Ethernet (definición del polinomio CRC-32, 0x04C11DB7).
- Kurose, J. F., & Ross, K. W. (2021). Computer Networking: A Top-Down Approach (8th ed.). Pearson. (Capítulo sobre la capa de enlace y detección/corrección de errores).
- Tanenbaum, A. S., & Wetherall, D. J. (2011). Computer Networks (5th ed.). Prentice Hall. (Códigos de Hamming y CRC en la capa de enlace de datos).

Repositorio

REPOSITORIO

GITHUB

